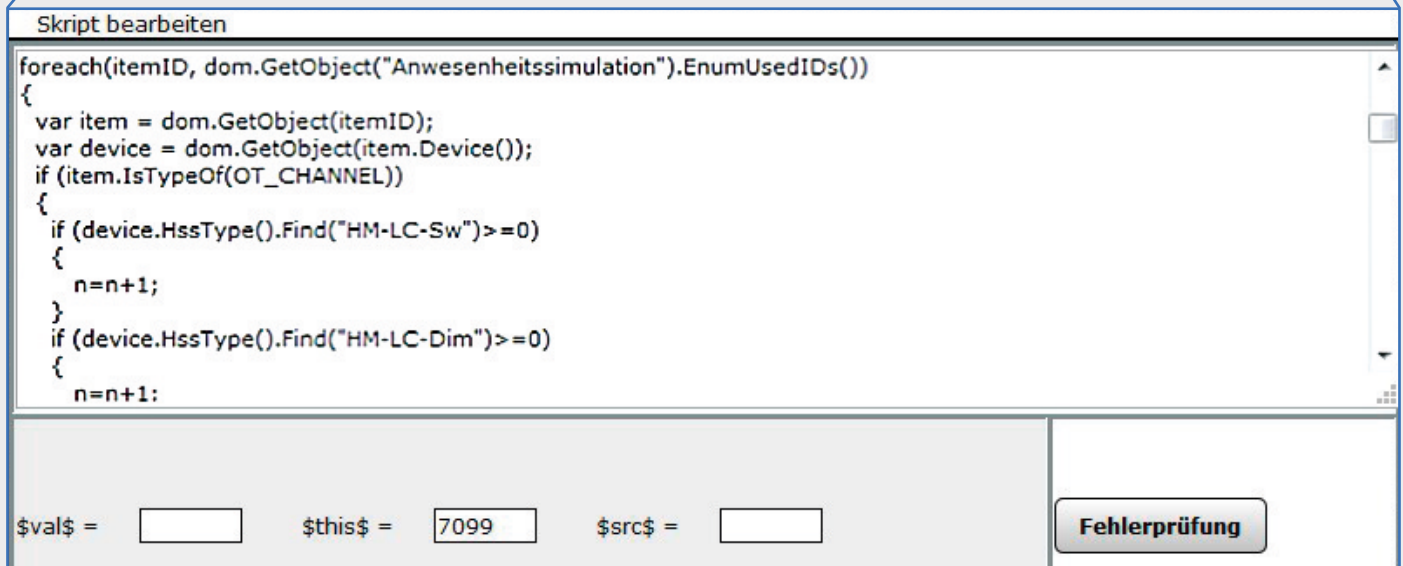
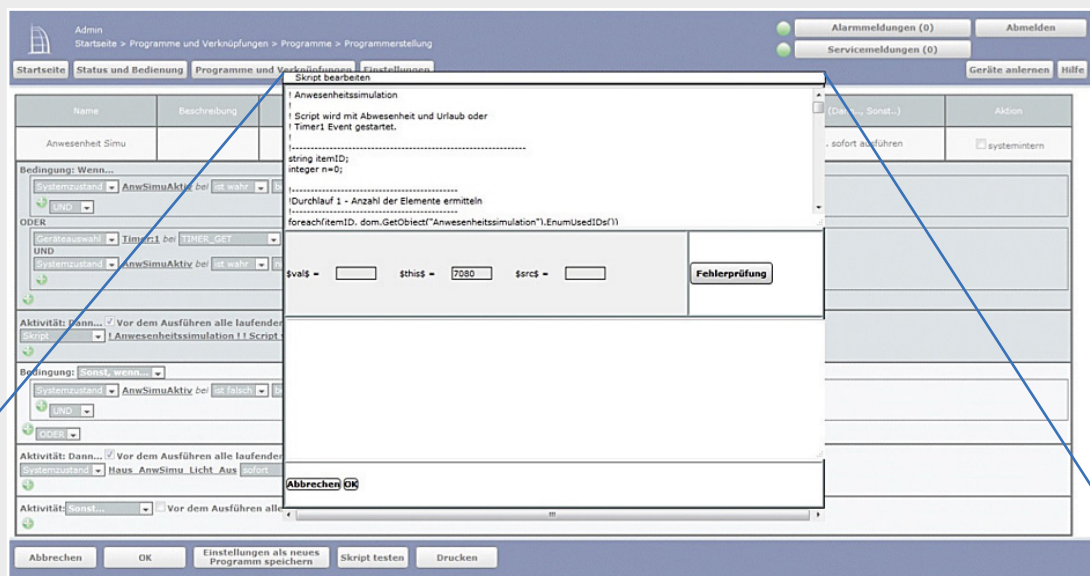




Homematic Scriptprogrammierung

Teil 5 – API-Kommunikation und Scripte



Im fünften Teil der Artikelserie über die Scriptprogrammierung beschäftigen wir uns mit der Kommunikation mit Geräten mit API-Schnittstellen und schreiben weitere Scripte.



Was ist eine API?

Frei ins Deutsche übersetzt bedeutet der aus dem englischen Sprachraum stammende Begriff „Application-Programming-Interface“ (API) Schnittstelle zur Anwendungsprogrammierung.

Eine API stellt in der Regel Bibliotheken bereit, deren Befehle Anfragen entgegennehmen, an die Hardware weiterleiten und diese Anfragen auch beantworten können (Wiedergabe von digitalen Inhalten).

Besonders bei der Wiedergabe von digitalen Inhalten hat der Begriff API im Bereich der Webdienste Bedeutung erhalten. Es ist hier möglich, über Anfragen an die API einen Inhalt an das aufrufende Programm zur weiteren Verwendung abzurufen.

Die API ist quasi das für einen Rechner nutzbare Äquivalent zu einem User-Interface. Daten werden von einer API meist in einem XML-Format ausgegeben.

Beispiel Wetter und Wettervorhersage

Die Seite wunderground.com liefert das aktuelle Wetter und die Wettervorhersage. Zunächst muss man sich auf der Seite (kostenlos) anmelden und erhält dann einen Schlüssel, mit dem die gewünschten Abfragen möglich sind. Es gibt zahlreiche weitere Seiten, die einen solchen Service bieten.

Die Abfrage bei Wunderground erfolgt nun z. B. folgendermaßen:

Abfrage des aktuellen Wetters in Berlin:

http://api.wunderground.com/api/Ihr_Schlüssel/conditions/lang:DE/q/Germany/Berlin.xml

Abfrage der 3-Tage-Wettervorhersage für Berlin:

http://api.wunderground.com/api/Ihr_Schlüssel/forecast/lang:DE/q/Germany/Berlin.xml

„Ihr_Schlüssel“ steht für den Schlüssel, den Sie nach der Registrierung auf Wunderground bekommen, „conditions“ ist das Schlüsselwort für das aktuelle Wetter, „forecast“ das für die 3-Tage-Wettervorhersage.

Die Endung .XML veranlasst die API von Wunderground, die Daten als XML-File zu senden. Die Stadt (hier Berlin) kann ebenfalls entsprechend Ihren Gegebenheiten aus einer großen Anzahl gewählt werden. Stöbern Sie die Seiten von Wunderground einmal durch.

Tipp: Wenn Sie sich bei dem Dienst angemeldet haben, können Sie das Ganze einfach mal ausprobieren, indem Sie einen der obigen Aufrufe (natürlich mit Ihrem Schlüssel anstelle von „Ihr_Schlüssel“) in die Adresszeile Ihres Browsers eingeben.





Für den Fall 1 (aktuelles Wetter) erhalten Sie beispielsweise ein File ähnlich diesem:

01	<response>
02	<version>0.1</version>
03	<termsOfService>http://www.wunderground.com/weather/api/d/terms.html</termsOfService>
04	<features><feature>conditions</feature></features>
05	<current_observation>
06	<image>
07	<url>http://icons.wxug.com/gra
08	<title>Weather Underground</tit
09	<link>http://www.wunderground.c
10	</image>
11	<display_location>
12	<full>Berlin, Germany</full>
13	<city>Berlin</city>
14	<state>BE</state>
15	<state_name>Germany</state_name
16	<country>DL</country>
17	<country_iso3166>DE</country_is
18	<zip>00000</zip>
19	<magic>46</magic>
20	<wmo>10389</wmo>
21	<latitude>52.52000046</latitude
22	<longitude>13.40999985</longitu
23	<elevation>45.1</elevation>
24	</display_location>
25	<observation_location>
26	<full>, </full>
27	<country_iso3166>DE</country_iso3166>
28	<elevation> ft</elevation>
29	</observation_location>
30	<observation_time>Last Updated on May 24, 8:30 AM CEST</observation_time>
31	<observation_time_rfc822>Wed, 24 May 2017 08:30:16 +0200</observation_time_rfc822>
32	<observation_epoch>1495607416</observation_epoch>
33	<local_time_rfc822>Wed, 24 May 2017 08:44:43 +0200</local_time_rfc822>
34	<local_epoch>1495608283</local_epoch>
35	<local_tz_short>CEST</local_tz_short>
36	<local_tz_long>Europe/Berlin</local_tz_long>
37	<local_tz_offset>+0200</local_tz_offset>
38	<weather>Overcast</weather>
39	<temperature_string>54 F (12 C)</temperature_string>
40	<temp_f>54</temp_f>
41	<temp_c>12</temp_c>
42	<relative_humidity>71</relative_humidity>
43	<wind_string>From the WNW at 13 MPH</wind_string>
44	<wind_dir>WNW</wind_dir>
45	<wind_mph>13</wind_mph>
46	<wind_gust_mph>0</wind_gust_mph>
47	<wind_kph>21</wind_kph>
48	<wind_gust_kph>0</wind_gust_kph>
49	<pressure_mb>1018.3</pressure_mb>
50	<pressure_in>30.07</pressure_in>
51	<dewpoint_string>44 F (7 C)</dewpoint_string>
52	<dewpoint_f>44</dewpoint_f>
53	<dewpoint_c>7</dewpoint_c>
54	<heat_index_string>NA</heat_index_string>
55	<heat_index_f>NA</heat_index_f>
56	<heat_index_c>NA</heat_index_c>
58	<windchill_c>10</windchill_c>
59	<feelslike_string>50 F (10 C)</feelslike_string>
60	<feelslike_c>10</feelslike_c>
61	<visibility_mi>10</visibility_mi>
62	<visibility_km>16</visibility_km>
63	</current_observation>
64	</response>

Rückgabe der angeforderten Daten von der API in Form einer XML-Datei.

XML (engl. Extensible Markup Language = Erweiterbare Auszeichnungssprache) ist eine Sprache für die Darstellung hierarchisch strukturierter Daten in Form einer Textdatei.

Sie wird für den plattformunabhängigen Austausch von Daten zwischen Computersystemen, vor allem über das Internet, verwendet.

Unten sind 3 Beispiele für Daten markiert, die die Temperatur in Fahrenheit, die Temperatur in °C und die relative Luftfeuchtigkeit liefern.

Temperatur in Fahrenheit

Temperatur in °C

Relative Luftfeuchtigkeit



Wie kommt man nun in der Homematic zu den Daten? Hier verwenden wir wieder die Befehle aus dem im vorhergehenden Artikel der Serie besprochenen CUxD Add-on:

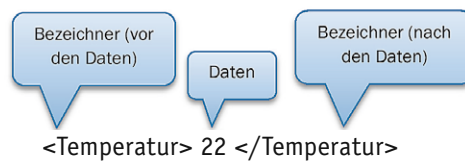
01	<code>var url="http://api.wunderground.com/api/Ihr_Schlüssel/conditions/lang:DE/q/Germany/Berlin.xml";</code>
02	<code>dom.GetObject("CuX System Exec:1.CMD_SETS").State("wget -q -O -" # url);</code>
03	<code>dom.GetObject("CuX System Exec:1.CMD_QUERY_RET").State(1);</code>
04	<code>string wetter_xml = dom.GetObject("CuX System Exec:1.CMD_RETS").State();</code>

Zeile 1 → schreibt die URL, mit der die API angesprochen werden soll, in die Variable `url`.

Zeile 2 → legt den auszuführende Befehl fest, indem er in den Datenpunkt `CMD_SETS` geschrieben wird, hier `"wget -q -O -" # url`

Zeile 3 → hier wird `CMD_QUERY_RET` auf 1 gesetzt, so kann anschließend mit `CMD_RETS` der Output (das gesuchte XML-File) abgefragt werden.

Zeile 4 → liest den Inhalt des XML-Files, das von der API zurückgegeben wird, in die Stringvariable `wetter_xml` ein. Nun müssen wir uns noch die Daten, die wir verwenden wollen, aus dem XML-File (dem String `wetter_xml`) „herausholen“. Ein XML-File ist immer so aufgebaut, dass Daten von einem Bezeichner eingeschlossen werden. Beispiel:



`<Temperatur>` sagt aus, dass Daten folgen, `</Temperatur>` beendet ein Datum. Dazwischen steht das eigentliche Datum, in diesem Fall 22 (°C). Eine Schachtelung ist auch möglich. Beispiel:

```
<Temperatur>
  <Tag> 22 </Tag>
  <Nacht> 12 </Nacht>
</Temperatur>
```

Die Homematic Scriptsprache bietet uns nun im Bereich der Stringverarbeitung über die Funktionen `.Length()`, `.Find()` und `.Substr()` Möglichkeiten, uns die Daten aus dem XML-File zu extrahieren. Folgender Dekodierblock erfüllt unsere Anforderungen:

05	<code>string suchwort = "<relative_humidity>";</code>
06	<code>integer suchwortlaenge = suchwort.Length();</code>
07	<code>integer wortposition = wetter_xml.Find(suchwort);</code>
08	<code>string suchwort = "</relative_humidity>";</code>
09	<code>integer wortendepos = wetter_xml.Find(suchwort);</code>
10	<code>string daten = wetter_xml.Substr((wortposition+suchwortlaenge+1), wortendepos- (wortposition+suchwortlaenge+1));</code>
11	<code>dom.GetObject("Luftfeuchtigkeit").State(daten);</code>

Wir nehmen nun an, das Zeichen `<` vor unserem Suchwort (dem Bezeichner) befindet sich an Position 55 in der Variablen `wetter_xml`. Schauen wir uns die Funktion der Programmzeilen an:

```
5556575859606162636465666768697071727374757677787980818283848586878889909192939495
<relative_humidity>71</relative_humidity>
```

Zeile 5: Den gesuchten Wert für die Luftfeuchtigkeit finden wir nach dem Suchwort `<relative humidity>`. Dieses wird in eine Stringvariable geschrieben.

Zeile 6: Hier wird die Länge des Suchwortes berechnet. Der Wert `suchwortlaenge` wird 18.

Zeile 7: Die Position des Suchwortes im String `wetter_xml` wird ermittelt: `wortposition` wird 55.

Zeile 8: Der die Daten abschließende String `</relative humidity>` wird in eine Stringvariable geschrieben.

Zeile 9: Die Position dieses abschließenden Strings wird gesucht: `wortendepos` wird 76.

Zeile 10: Der Datenstring beginnt nun an der Position `wortposition+suchwortlaenge+1 = 74`.

Über den Beginn und die Länge des Datenstrings wird dieser nun über die SubString-Funktion ermittelt und in die Stringvariable `daten` geschrieben: `daten = "71"`.

Die Länge des Datenstrings berechnet sich aus `wortendepos-(wortposition+suchwortlaenge+1) = 2`.

Zeile 11: Das Ergebnis (die Luftfeuchtigkeit) wird in die (vorher anzulegende) Zentralenvariable (Typ Zeichenkette) `Luftfeuchtigkeit` geschrieben. Soll die Variable den Typ Zahl haben, heißt die Programmzeile 11:

chreibt die URL, mit der die API angesprochen werden soll, in die Variable `u`

Was wir jetzt noch machen müssen, ist, Zentralenvariablen anzulegen, in die wir die für uns interessanten Daten schreiben wollen, und den obigen Scriptblock für unsere gewünschten Daten mit den jeweiligen Schlüsselwörtern zu wiederholen sowie das Script bei Bedarf zu starten.



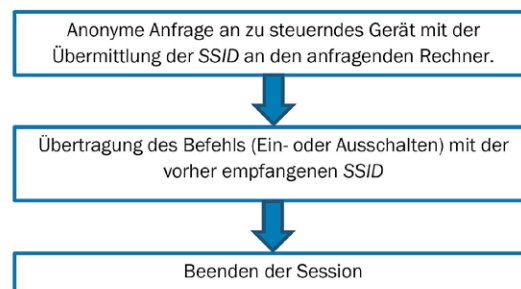
Hier noch der Dekodierblock als Dreizeiler. Es werden nur noch zwei Variablen verwendet, dafür wird aber die Zeile 6 extrem unübersichtlich.

05	<code>string suchwort = "<relative_humidity>";</code>
06	<code>string daten = wetter_xml.Substr((wetter_xml.Find(suchwort)+suchwort.Length()+1) , wetter_xml.Find(suchwort.Substr(0,1) # "/" #suchwort.Substr(1, suchwort.Length()-1)) - (wetter_xml.Find(suchwort)+suchwort.Length()+1));</code>
07	<code>dom.GetObject("Luftfeuchtigkeit").State(daten);</code>

Beispiel Internet-Receiver-Ansteuerung mit Homematic

Neben der Bedienung über Infrarot sind die meisten Geräte aus dem Audio-/Video-Bereich heute über TCP/IP, also über Netzwerk steuerbar. Dies zeigt sich in der stetig wachsenden Zahl von Apps für Smartphones und für Tablets, mit denen diese Geräte dann steuerbar sind. Die Steuerung erfolgt auch hier über eine API. Im Folgenden Beispiele für die Ansteuerung eines Chips von Frontier Silicon, der in einigen der heute erhältlichen Audiogeräte, die Internetradio- und/oder streamingfähig sind, eingesetzt wird. Für andere Geräte ist natürlich die gerätespezifische API zu verwenden. Die Daten dazu findet man zumeist im Internet.

Ein Unterschied zum vorhergehenden Beispiel ist der, dass sich für die Kommunikation hier zuerst ein Teilnehmer mit dem Audiogerät quasi „verbinden“ muss. Dies geschieht durch eine erste Anfrage, auf die hin das Audiogerät mit einer sogenannten SSID antwortet. Die SSID ist ein „Service Set Identifier“, also eine Identifikationsnummer, die den Sender einer Anfrage eindeutig kennzeichnet (wir kennen diese SSIDs beispielsweise auch von WLAN-Netzwerken). In diesem Fall hier wird die SSID nach einer ersten „anonymen“ Anfrage dem anfragenden Teilnehmer mitgeteilt. Alle weiteren Anfragen werden danach unter Zuhilfenahme dieser SSID gemacht, solange, bis die Verbindung geschlossen wird. Eine „Session“ zum Ein- oder Ausschalten besteht somit z. B. aus folgenden Datenkommunikationen (aus folgendem Ablauf):



Folgende API-Befehle werden in den folgenden Beispielen verwendet:

1.) SSID anfordern

01	<code>http://IP-ADRESSE/fsapi/CREATE_SESSION?pin=1234</code>
----	--

Auf diese Anfrage hin sendet das Audiogerät z. B. folgende Antwort:

```

<fsapiResponse>
  <status>FS_OK</status>
  <sessionId>237288973</sessionId>
</fsapiResponse>
  
```

Wie schon im Beispiel mit der Abfrage der Wettervorhersage gezeigt, beginnen die relevanten Daten immer mit einem Bezeichner der Form <Bezeichner>, dann kommen die Daten, danach die Ende-Kennung für diesen Bezeichner, also </Bezeichner>. Der Bezeichner ist hier die „sessionId“.

2.) Gerät einschalten (Power ON)

02	<code>http://IP-ADRESSE/fsapi/SET/netRemote.sys.power?pin=1234&sid=SID-NUMMER&value=1</code>
----	--

3.) Gerät ausschalten (Power OFF)

03	<code>http://IP-ADRESSE/fsapi/SET/netRemote.sys.power?pin=1234&sid=SID-NUMMER&value=0</code>
----	--

4.) Lautstärke des Gerätes abfragen

04	<code>http://IP-ADRESSE/fsapi/GET/netRemote.sys.audio.volume?pin=1234&sid=SID-NUMMER</code>
----	---

5.) Lautstärke des Gerätes setzen

05	<code>http:// IP-ADRESSE/fsapi/SET/netRemote.sys.audio.volume?pin=1234</code>
----	---



Die gesamte API findet sich hier: <https://github.com/flammy/fsapi/blob/master/FSAPI.md>

In den Scripten werden folgende (globale) Zentralenvariablen verwendet:

Audio_IP = IP-Adresse des Receivers
 Audio_PIN = Pin-Nr. des Receivers
 AUDIO_VOL_BIT1 = Bit 1 für Lautstärkeeinstellung
 AUDIO_VOL_BIT2 = Bit 2 für Lautstärkeeinstellung
 AUDIO_VOL_BIT3 = Bit 3 für Lautstärkeeinstellung
 AUDIO_VOL_BIT4 = Bit 4 für Lautstärkeeinstellung
 AUDIO_VOL_BIT5 = Bit 5 für Lautstärkeeinstellung
 AUDIO_VOL_SET = Lautstärke entsprechend Bitmuster Bit 1 ... Bit 5 setzen

Script für die Funktion „SSID anfordern“

01	!-----
02	! benoetigte (Zentralen-) Variablen:
03	!
04	! AUDIO_IP TEXT Ip-Adresse des Receivers
05	! AUDIO_PIN TEXT pin des Receivers (default: 1234)
06	!-----
07	!-----
08	!einloggen und ssid
09	!-----
10	var url = "http://" # dom.GetObject("AUDIO_IP").State() # "/fsapi/CREATE_SESSION?pin=" # dom.GetObject("AUDIO_PIN").State();
11	dom.GetObject("CUxD.CUX2801001:1.CMD_SETS").State("wget -q -O - '"url#'"");
12	dom.GetObject("CUxD.CUX2801001:1.CMD_QUERY_RET").State(1);
13	string s = dom.GetObject("CUxD.CUX2801001:1.CMD_RETs").State();
14	integer word_position = s.Find("sessionId") + 10;
15	string daten = s.Substr(word_position ,s.Length()-word_position);
16	word_position = daten.Find("/sessionId");
17	string ssid = daten.Substr(0,word_position-1);

Die Session-ID ist nun in der (im Script lokalen) Variablen SSID gespeichert.

Script für die Funktion „Gerät einschalten“

18	!-----
19	!Einschalten
20	!-----
21	url="http://" # dom.GetObject("AUDIO_IP").State() # "/fsapi/SET/netRemote.sys.power?pin=" # dom.GetObject("AUDIO_PIN").State()#"&sid="#" ssid #"&value=1";
22	dom.GetObject("CUxD.CUX2801001:1.CMD_SETS").State("wget -q -O - '"url#'"");
23	dom.GetObject("CUxD.CUX2801001:1.CMD_QUERY_RET").State(1);
24	string s = dom.GetObject("CUxD.CUX2801001:1.CMD_RETs").State();

Im Befehl in Zeile 21 (Power einschalten) wird die oben ermittelte SSID, die in der Variablen SSID gespeichert ist, verwendet.

In der Zeile 24 wird der Return-String, den das zu schaltende Gerät zurückgibt, eingelesen. Hier kann z. B. bei Misserfolg aufgrund der Ursache entschieden werden, wie weiter vorgegangen wird. Machen Sie hier Ihre eigenen Versuche.

Script für die Funktion „Gerät ausschalten“

18	!-----
19	!Ausschalten
20	!-----
21	url="http://" # dom.GetObject("AUDIO_IP").State() # "/fsapi/SET/netRemote.sys.power?pin=" # dom.GetObject("AUDIO_PIN").State()#"&sid="#" ssid #"&value=0";
22	dom.GetObject("CUxD.CUX2801001:1.CMD_SETS").State("wget -q -O - '"url#'"");
23	dom.GetObject("CUxD.CUX2801001:1.CMD_QUERY_RET").State(1);
24	string s = dom.GetObject("CUxD.CUX2801001:1.CMD_RETs").State();

In der Zeile 31 wird der Return-String, den das zu schaltende Gerät zurückgibt, eingelesen. Hier kann z. B. bei Misserfolg aufgrund der Ursache entschieden werden, wie weiter vorgegangen wird. Machen Sie hier Ihre eigenen Versuche.



32	!-----
33	!Session beenden
34	!-----
35	url = "http://" # dom.GetObject("AUDIO_IP").State() # "/fsapi/DELETE_SESSION?pin=" # dom.GetObject("AUDIO_PIN").State()#"&sid=" # ssid;
36	dom.GetObject("CUXD.CUX2801001:1.CMD_SETS").State("wget -q -O - '"#url#"");
37	dom.GetObject("CUXD.CUX2801001:1.CMD_QUERY_RET").State(1);
38	string s = dom.GetObject("CUXD.CUX2801001:1.CMD_RET").State();

Script für die Funktion „Session beenden“

Auch hier wird in der Zeile 38 der Return-String, den das zu schaltende Gerät zurückgibt, eingelesen.

Anmerkung: Die obigen Scripte sind als ein gesamtes Script zu sehen, wobei für die Aktion entweder das Script für „Gerät einschalten“ oder das Script für „Gerät ausschalten“ verwendet wird. Zur besseren Erklärung sind die einzelnen Teile des Scriptes separat dargestellt.

Script für die Funktion „Lautstärke einstellen“

Die Lautstärke wird hier durch eine Binär-Codierung von fünf Zentralenvariablen AUDIO_VOL_BIT1 ... AUDIO_VOL_BIT5 gesetzt. Es kann selbstverständlich auch eine Systemvariable vom Typ Zahl verwendet werden.

Die Lautstärke ergibt sich aus folgender Wertetabelle:

Bit 5 (Wert 16)	Bit 4 (Wert 8)	Bit 3 (Wert 4)	Bit 2 (Wert 2)	Bit 1 (Wert 1)	Lautstärke
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	2
0	0	0	1	1	3
0	0	1	0	0	4
...	...	...	...	...	...
1	1	1	1	1	31

Das Script:

01	!-----
02	! benoetigte Variablen:
03	!-----
04	! AUDIO_IP TEXT IP-Adresse des Receivers
05	! AUDIO_PIN TEXT pin des Receivers (default: 1234)
06	! AUDIO_VOL_BIT1 BOOL Bit 1 fuer Lautstaerke
07	! AUDIO_VOL_BIT2 BOOL Bit 2 fuer Lautstaerke
08	! AUDIO_VOL_BIT3 BOOL Bit 3 fuer Lautstaerke
09	! AUDIO_VOL_BIT4 BOOL Bit 4 fuer Lautstaerke
10	! AUDIO_VOL_BIT5 BOOL Bit 5 fuer Lautstaerke
11	! AUDIO_VOL_SET BOOL Lautstaerke setzen (cmd)
12	!-----
13	!-----
14	!Wert aus den Bits berechnen
15	!-----
16	var wert = 0;
17	if (dom.GetObject("AUDIO_VOL_BIT1").State() == true) { wert = wert + 1;}
18	if (dom.GetObject("AUDIO_VOL_BIT2").State() == true) { wert = wert + 2;}
19	if (dom.GetObject("AUDIO_VOL_BIT3").State() == true) { wert = wert + 4;}
20	if (dom.GetObject("AUDIO_VOL_BIT4").State() == true) { wert = wert + 8;}
21	if (dom.GetObject("AUDIO_VOL_BIT5").State() == true) { wert = wert + 16;}
22	!-----
23	!einloggen und ssid
24	!-----
25	var url = "http://" # dom.GetObject("AUDIO_IP").State() # "/fsapi/CREATE_SESSION?pin=" # dom.GetObject("AUDIO_PIN").State();
26	dom.GetObject("CUXD.CUX2801001:1.CMD_SETS").State("wget -q -O - '"#url#"");
27	dom.GetObject("CUXD.CUX2801001:1.CMD_QUERY_RET").State(1);
28	string s = dom.GetObject("CUXD.CUX2801001:1.CMD_RET").State();
29	integer word_position = s.Find("sessionId") + 10;



30	string daten = s.Substr(word_position ,s.Length()-word_position);
31	word_position = daten.Find("/sessionId");
32	string ssid =daten.Substr(0,word_position-1);
33	!-----
34	!aktuelle Lautstaerke holen
35	!-----
36	url="http://192.168.1.57/fsapi/GET/netRemote.sys.audio.volume?pin=1234&sid="#sid;
37	dom.GetObject("CUxD.CUX2801001:1.CMD_SETS").State("wget -q -O - '"url#"");
38	dom.GetObject("CUxD.CUX2801001:1.CMD_QUERY_RET").State(1);
39	string s = dom.GetObject("CUxD.CUX2801001:1.CMD_RETs").State();
40	!-----
41	!Lautstaerke setzen
42	!-----
43	url="http://" # dom.GetObject("AUDIO_IP").State() # "/fsapi/SET/netRemote.sys.audio.volume?pin=" # dom.GetObject("AUDIO_PIN").State()#"&sid="#" ssid #"&value="#wert.ToString();
44	dom.GetObject("CUxD.CUX2801001:1.CMD_SETS").State("wget -q -O - '"url#"");
45	dom.GetObject("CUxD.CUX2801001:1.CMD_QUERY_RET").State(1);
46	string s = dom.GetObject("CUxD.CUX2801001:1.CMD_RETs").State();
47	!-----
48	!Session beenden
49	!-----
50	url = "http://" # dom.GetObject("AUDIO_IP").State() # "/fsapi/DELETE_SESSION?pin=" # dom.GetObject("AUDIO_PIN").State()#"&sid=" # ssid;
51	dom.GetObject("CUxD.CUX2801001:1.CMD_SETS").State("wget -q -O - '"url#"");
52	dom.GetObject("CUxD.CUX2801001:1.CMD_QUERY_RET").State(1);
53	string s = dom.GetObject("CUxD.CUX2801001:1.CMD_RETs").State();
54	

Die CUxD-Befehle müssen selbstverständlich den eigenen Gegebenheiten angepasst werden!

In Zeile 17 bis 21 wird der zu setzende Lautstärkewert aus den fünf Lautstärkebits berechnet und in die Variable „Wert“ geschrieben.

In den Zeilen 23 bis 32 erfolgt der Login mit der Rückgabe der SSID.

Die Zeilen 33 bis 39 (Einlesen des aktuellen Lautstärkewertes) sind für dieses Script eigentlich irrelevant, sie wurden für spätere Erweiterungen der Vollständigkeit halber hinzugefügt.

Die Zeilen 40 bis 46 setzen den neuen Lautstärkewert, in den Zeilen 47 bis 53 erfolgt das Schließen der Session.

Anmerkungen zu den Scripten:

Bei einer „perfekten“ Programmierung sollten die Rückgaben, wie schon erwähnt, nach dem Absetzen der Befehle geprüft werden.

Weiterhin sollte ein übergebener Lautstärkewert auf Plausibilität überprüft werden, d. h., es sollte vermieden werden, dass ein unsinniger Wert an das Audiogerät übergeben wird (z. B. kleiner als Minimalwert oder größer als Maximalwert). Hier gibt es genügend „Spielraum“ für eigene Versuche.

Ausblick

Im sechsten Teil dieser Reihe werden wir uns mit weiteren TCP/IP-Kommunikationen beschäftigen und ein paar kleinere nützliche Scripte zeigen. **ELV**

KOMMUNIKATION